DriftBench-TS: A Deployment-Oriented Benchmark for Time Series Forecasting Under Distribution Shift

Sachin*, Madhan Karthikeyan*, Madhu*
*Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

***Abstract*—Time series forecasting models are embedded in high-stakes decision systems spanning energy load scheduling, road traffic management, financial trading, and retail demand planning. Despite the proliferation of sophisticated forecasting architectures, prevailing evaluation practices rely on a single static train–test split that implicitly assumes stationarity and ignores the realities of long-running deployment: data distributions evolve over time, models require periodic re-training, and operational constraints impose hard limits on computational expenditure.**

We present *DriftBench-TS*, an open-source, deployment-oriented benchmark and simulation framework for rigorous evaluation of forecasting models under distribution shift. The framework supports four real-world public datasets spanning finance, energy, and mobility domains; a heterogeneous model zoo covering statistical baselines, gradient-boosting ensembles, recurrent networks, and feature-engineered models; and three configurable maintenance policies—no re-training, fixed-interval re-training every ten evaluation steps, and adaptive drift-triggered re-training governed by Kolmogorov-Smirnov test monitoring—with all events logged for post-hoc causal analysis.

Through a comprehensive experimental study across 72 distinct configuration runs, we characterise how model families and maintenance strategies trade off forecast accuracy, temporal robustness, and cumulative computational cost across domains with distinct drift characteristics. Our results reveal that model selection is dataset-dependent: LightGBM achieves 92% improvement on volatile energy data, while TSMixer outperforms all other models by 95% on financial/oil price data. Fixed-interval retraining provides consistent improvements, while adaptive strategies require domain-specific threshold tuning. Maintenance policy selection is often as consequential as architecture choice.

***Index Terms*—time series forecasting, distribution shift, concept drift, data drift, benchmarking, MLOps, deployment simulation, model monitoring, temporal robustness, re-training policy**

I. Introduction

Time series forecasting underpins critical decisions across a wide range of application domains. Energy utilities rely on short-term load forecasts to balance supply and demand; logistics operators use traffic-speed predictions to reroute fleets in near real time; banks embed volatility forecasts inside automated trading strategies; and retailers use demand models to drive inventory replenishment at scale. The societal and economic stakes attached to these forecasts are substantial, and errors translate directly into grid instability, increased emissions, financial loss, or supply-chain disruption.

Recent years have produced a remarkable proliferation of forecasting architectures. Classical methods such as ARIMA and exponential smoothing have been joined by gradient-boosting machines (LightGBM, XGBoost), which excel at tabular, feature-engineered representations of time series, and by a diverse family of deep learning models— recurrent networks (LSTM, GRU), Transformer-based architectures (Informer, Autoformer, PatchTST), and multi-layer perceptrons specialised for temporal data (TSMixer, DLinear) [**?**], [**?**].

However, evaluation practices have not kept pace with architectural advances. The dominant paradigm remains static one-shot evaluation: a model is trained on a historical window and assessed on a held-out future segment, with a single aggregate error metric—typically Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE)—used to declare a winner. This protocol is convenient but inadequate because it makes two implicit assumptions that rarely hold in practice. First, it assumes *stationarity*: that the joint distribution of inputs and targets is the same at training time and at deployment time. Second, it assumes *finality*: that a single training run is sufficient for the entire operational lifetime of the model.

In production, neither assumption holds. Time series data are generated by non-stationary processes driven by seasonal cycles, economic shocks, policy interventions, sensor degradation, and evolving user behaviour. The resulting *concept drift* causes models tuned on historical windows to degrade in ways that static benchmarks cannot detect. Engineers must make ongoing decisions about when and how to re-train models, what drift signals warrant an emergency update, and how to balance accuracy against the computational and organisational cost of frequent retraining. These decisions are MLOps concerns, yet they remain largely disconnected from academic forecasting research.

This paper introduces **DriftBench-TS**, a deployment-oriented benchmark and open-source framework that makes distribution shift and maintenance policy explicit, first-class components of forecasting evaluation. The framework fills the gap between static academic benchmarks and the operational reality of deployed forecasting systems. Our key contributions are:

- A unified *deployment simulation engine* that emulates streaming operation via rolling training windows, configurable forecast horizons and step sizes, and pluggable maintenance policies, all reproducible from a single YAML configuration file.
- A *model zoo* covering six model families—naïve baselines, seasonal naïve, random forests, gradient-boosting, recurrent networks, and MLP mixers—all exposed through a

common `fit`/`predict`/`update` interface.

- *Drift detection and monitoring modules* that apply the Kolmogorov–Smirnov test to sliding windows of target residuals, logging drift events for post-hoc analysis.
- A *comprehensive experimental study* spanning 72 configuration runs across 4 datasets, 6 models, and 3 maintenance policies, quantifying accuracy–robustness–cost trade-offs.
- An *open-source implementation* with YAML-driven configuration, reproducible experiment runners, and analysis tools.

The remainder of this paper is structured as follows. Section II reviews related work. Section III formalises the problem and deployment simulation. Section IV describes the system design. Section V details the experimental setup and metrics. Section VI presents results. Section VII discusses implications and limitations, and Section VIII concludes.

II. Related Work

A. Time Series Forecasting Benchmarks

Benchmarking for time series forecasting has evolved considerably over the past decade. Early efforts relied on small, well-known collections such as the M-competitions and the UCR/UEA archive, which established community norms but were limited in scale and domain diversity. More recent large-scale initiatives have raised the bar significantly. Weather2K aggregates multivariate meteorological observations from over two thousand ground stations, introducing realistic missingness and spatio-temporal heterogeneity as first-class challenges [**?**]. LargeST provides five years of five-minute-resolution traffic speed data from 8,600 sensors across California, complete with graph topology metadata and baseline implementations for deep spatio-temporal models [**?**]. TSLib and BasicTS unify implementations of dozens of deep learning architectures across shared preprocessing pipelines, enabling controlled comparisons that isolate architectural differences from data handling artefacts [**?**], [**?**].

These contributions share a common methodological limitation: models are trained once and evaluated on a fixed test window, with no mechanism to study what happens to performance as time progresses and data distributions shift. This design choice privileges architectural comparison over operational realism.

B. Concept Drift and Temporal Robustness

Concept drift—the phenomenon whereby the statistical properties of the target variable change over time—has been studied extensively in the streaming classification literature. Gama et al. formalised drift taxonomies covering sudden, gradual, incremental, and recurrent drift, and proposed ensemble-based adaptation algorithms such as ADWIN and DDM [**?**]. However, this foundational work predominantly addresses classification tasks with explicit class-label feedback that is not available in regression and forecasting settings.

In the broader machine learning systems literature, recent work emphasises the need for benchmarks that explicitly model data and workload drift to evaluate learned components under realistic dynamics [**?**]. For time series specifically, most publicly available forecasting benchmarks either do not annotate drift windows explicitly or provide no tooling to simulate deployment scenarios such as rolling re-training or adaptive update triggers. DriftBench-TS is designed specifically to fill this operational void.

C. MLOps and Deployment-Focused Evaluation

The emerging discipline of MLOps emphasises the full life cycle of machine learning systems: data versioning, experiment tracking, model monitoring, re-training orchestration, and production governance. Industrial platforms such as Azure Machine Learning and MLflow provide infrastructure for drift detection and pipeline automation, but they do not supply domain-specific benchmarks that allow systematic comparison of forecasting models and maintenance strategies under controlled experimental conditions [**?**]. Open-source projects such as TSBenchmark have begun to bridge this gap, but they focus primarily on tabular classification rather than temporal forecasting, and they do not provide the rolling deployment simulation or maintenance-policy abstraction that DriftBench-TS offers [**?**]. DriftBench-TS is, to our knowledge, the first open-source benchmark to make deployment simulation, configurable maintenance policies, and drift-correlated error analysis co-equal components of time series forecasting evaluation.

III. Problem Definition

A. Notation and Forecasting Task

Let $\mathcal{T} = \{1, 2, \ldots, T\}$ index a discrete time axis. At each time step $t \in \mathcal{T}$, we observe a covariate vector $x_t \in \mathbb{R}^d$ and a target vector $y_t \in \mathbb{R}^m$. Covariates x_t may include calendar features (hour-of-day, day-of-week, public holidays), weather measurements, or exogenous series known at prediction time; targets y_t represent the quantities to be forecast (e.g., energy load, traffic speed, oil price).

Given a history window of length H, the forecasting task at time t is to produce a forecast $\hat{y}_{t+1:t+F} = f_\theta(x_{t-H+1:t},\, y_{t-H+1:t})$ over the next F steps, where f_θ is a model parameterised by weights θ and F is the forecast horizon.

B. Distribution Shift Taxonomy

In the static evaluation setting, the joint data distribution is treated as stationary: $P_{\text{train}}(x, y) = P_{\text{test}}(x, y)$. Deployment invalidates this assumption. We distinguish three operationally relevant forms of distribution shift. *Covariate shift* occurs when the marginal input distribution changes—$P_{\text{train}}(x) \neq P_{\text{deploy}}(x)$—while the conditional $P(y \mid x)$ remains stable. *Label shift* refers to a change in the marginal target distribution, $P_{\text{train}}(y) \neq P_{\text{deploy}}(y)$, as in sustained demand growth or a step-change in baseline energy consumption. *Concept drift*, in the strict sense, refers to a change in the conditional $P(y \mid x)$, meaning the functional relationship between covariates and targets evolves over time. In practice, deployed forecasting systems face a mixture of all three forms. DriftBench-TS

monitors both covariate and target distributions independently, logging divergence scores and correlating them with forecast error trajectories.

C. Deployment Simulation

We model the deployment life cycle as a discrete sliding-window process parameterised by the tuple $(\mathcal{D}, f_\theta, H, F, S, \Pi, \Delta)$, where $\mathcal{D}$ is the dataset, f_θ is the chosen model, H and F are the history and horizon lengths, S is the evaluation step size, Π is the maintenance policy, and Δ is the drift detection configuration.

D. Maintenance Policies

No Retrain (Π_0): The model is trained once on an initial window and never updated. This lower-bound baseline requires zero ongoing computation but degrades as drift accumulates. Implemented by setting `min_steps_between_retrain: 999999` in the configuration.

Fixed-Interval Retrain (Π_K): The model is retrained every K evaluation steps using the most recent H observations. In our experiments, $K = 10$ for all datasets, implemented via `policy: fixed_schedule` with `retrain_every_n_steps: 10`.

Adaptive Retrain (Π_Δ): A drift detector monitors divergence between the reference distribution and the current observation window. When divergence exceeds a threshold of 0.5 (`drift_threshold: 0.5`), retraining is triggered, subject to a minimum gap of 3 steps (`min_steps_between_retrain: 3`) and requiring 2 consecutive detections (`require_consecutive: 2`) with exponential decay (`consecutive_decay_rate: 0.5`) to suppress transient false alarms.

IV. System Design

A. Architecture Overview

DriftBench-TS is decomposed into four layers designed for modularity and extensibility: (1) *dataset adapters* that ingest and normalise heterogeneous public datasets through a unified schema; (2) a *deployment simulator* that orchestrates the rolling evaluation loop, managing training windows, retraining events, and drift detector state; (3) a *model zoo* that wraps diverse forecasting algorithms behind a common interface; and (4) *evaluation and reporting* components that compute metrics and log events.

B. Dataset Adapters

DriftBench-TS ships adapters for four public datasets, chosen to cover three domains with distinct drift characteristics. All datasets are downsampled to a common operational frequency before use, and all experiments use a fixed random seed of 42.

Brent Crude Oil (Finance). Daily spot prices for Brent crude oil, sourced from the U.S. Energy Information Administration. For deployment simulation, data are downsampled to weekly cadence (`downsample_freq: w`), with a history window of 60 days (`history_window_days: 60`), a forecast horizon of 168 hours (one week ahead), and a step size of 168 hours.

TABLE I: Datasets included in DriftBench-TS.

Dataset	Domain	#Entities	Frequency	Horizon
Brent	Finance	1	Weekly	1 week
WTI	Finance	1	Weekly	1 week
Electricity	Energy	5 sampled	Hourly	24 h
Traffic	Mobility	1 sampled	Hourly	24 h

West Texas Intermediate Crude Oil (Finance). Daily WTI spot prices, structurally correlated with Brent but exhibiting distinct spread dynamics. The deployment configuration is identical to Brent.

Electricity Demand (Energy). Hourly electricity consumption from the UCI Electricity Load Diagrams dataset, covering 320 consumers [**?**]. The adapter samples 5 entities for each run at hourly resolution, with a history window of 30 days, a forecast horizon of 24 hours, and a step size of 24 hours.

Road Traffic Speed (Mobility). Hourly aggregated traffic speeds from the METR-LA dataset, comprising loop detectors on highways [**?**]. The adapter samples 1 entity per run at hourly resolution, with a history window of 30 days, a forecast horizon of 24 hours, and a step size of 24 hours.

Table I summarises the key characteristics.

C. Model Zoo

The model zoo exposes all algorithms through a uniform Python interface: `fit(X_train, y_train)` trains or retrains the model; `predict(X_test)` returns a forecast of length F.

Naïve Baseline. Copies the last observed value as the forecast. Zero parameters, $\mathcal{O}(1)$ training cost.

Seasonal Naïve. Copies the last complete seasonal cycle with a configured season length of 24 steps.

Random Forest (RF). A bagged ensemble of 50 decision trees with max depth 8, trained on a lag-feature matrix with 3 lags.

LightGBM. A gradient-boosted decision tree ensemble with 50 estimators, max depth 5, learning rate 0.1, and 3 lag features.

LSTM. A two-layer Long Short-Term Memory network with hidden size 32, ingesting sequences of 24 steps, trained for 20 epochs using PyTorch.

TSMixer. An MLP-based architecture using Ridge Regression with rich temporal features including: time-based features (hour, day, month, cyclical encodings), lag features (1-48), rolling statistics (mean, std). This approach captures similar patterns to the original TSMixer architecture without requiring GPU resources.

Table II summarises the model configurations.

D. Drift Detection and Monitoring

The drift monitoring subsystem computes a divergence score comparing the current observation window against the reference window used in the most recent training event.

The *Kolmogorov–Smirnov (KS) test* is applied to empirical CDFs of residuals, measuring non-parametric distribution discrepancy. The two-sample KS statistic $D = \sup_x |F_1(x) - F_2(x)|$ provides a distribution-free measure of change.

TABLE II: Model zoo: families, key hyperparameters, and complexity.

Model	Key Hyperparameters	Complexity
Naïve	—	$\mathcal{O}(1)$
Seasonal Naïve	Season = 24	$\mathcal{O}(1)$
Random Forest	Trees= 50, depth= 8	$\mathcal{O}(T \log T)$
LightGBM	Trees= 50, depth= 5	$\mathcal{O}(T \log T)$
LSTM	Layers= 2, hidden= 32	$\mathcal{O}(TH^2)$
TSMixer	Ridge + 48 lags	$\mathcal{O}(T^2 d)$

The adaptive policy triggers retraining when the drift score exceeds the configured threshold of 0.5, subject to the consecutive detection guard described in Section III-D.

V. Experimental Setup

A. Evaluation Protocol

For each of the 72 configuration combinations (4 datasets $\times$ 6 models $\times$ 3 policies), DriftBench-TS executes a rolling deployment procedure. An initial training corpus is assembled from the earliest H time steps and the model is trained with seed 42. The simulator advances in steps of size S, executing: (1) assembling a training window; (2) retraining if prescribed by policy; (3) generating forecasts; (4) computing point forecast errors; and (5) updating drift statistics. This continues until fewer than F future observations remain.

All experiments use seed 42. No further tuning was performed during the deployment window to avoid look-ahead bias.

B. Metrics

We evaluate forecasts using three standard metrics. Mean Absolute Error:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{1}$$

Root Mean Squared Error:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2} \tag{2}$$

Symmetric Mean Absolute Percentage Error:

$$\text{sMAPE} = \frac{200}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \tag{3}$$

Temporal robustness: Standard deviation of per-window MAE values.

Maintenance cost: Total re-training events and cumulative wall-clock training time.

C. Implementation Details

The framework is implemented in Python with dependencies: pandas, NumPy, scikit-learn, LightGBM, PyTorch, and Matplotlib. Configuration files are in YAML format; a single command reproduces any experimental run.

Hardware: Experiments conducted on an NVIDIA RTX 3050 GPU, Intel Core i5-12500H CPU, and 16 GB RAM.

VI. Results

A. Model Performance by Dataset

Table III presents MAE results across all datasets, models, and strategies.

B. Key Findings

1. Model selection is dataset-dependent. LightGBM achieves 90% MAE reduction on volatile electricity data (1495 $\rightarrow$ 120), while TSMixer outperforms all other models by 96% on financial/oil price data (1.81 vs 35.76 for Naive on Brent no-retrain).

2. Fixed retraining is most consistently effective. Across all models and datasets, fixed-interval retraining with $K = 10$ provides the best accuracy improvements at moderate compute cost.

3. Adaptive retraining excels on low-drift data. Our analysis reveals that adaptive retraining outperforms fixed retraining on financial/oil price datasets (Brent, WTI) where drift is gradual and periodic. LightGBM achieves 45% better MAE with adaptive retraining on Brent (4.00 vs 7.27) and 38% better on WTI (3.61 vs 5.85). However, on high-volatility datasets (electricity, traffic), fixed retraining remains superior due to consistent drift patterns.

4. TSMixer validates as a distinct model. TSMixer produces results significantly different from Naive baselines: 61% better on electricity, 95% better on Brent, 94% better on WTI.

C. Accuracy–Cost Trade-offs

LightGBM with fixed retraining dominates on volatile datasets due to its $\mathcal{O}(T \log T)$ training complexity. TSMixer achieves best accuracy on financial data where its feature engineering captures periodic patterns effectively.

D. Retraining Strategy Comparison

Table V compares the three maintenance strategies across datasets using LightGBM and TSMixer models.

Key observation: Fixed-interval retraining provides consistent improvements across datasets. Adaptive retraining also triggers effectively for most models (519 retrains on electricity, 494 on traffic, 11 on Brent), showing the KS detector is working when properly configured.

Insight: Adaptive retraining excels on low-drift financial datasets (Brent, WTI) with 38-45% improvement, while TSMixer on electricity shows 11.9% improvement even with fewer retrains.

E. Ablation Study: Retraining Interval

We evaluate the effect of varying the fixed retraining interval K on electricity data with LightGBM (Table VI).

Analysis: $K = 10$ provides a good trade-off between accuracy and compute cost. $K = 1$ yields only 17% MAE improvement over $K = 10$ but requires 10$\times$ more retrains. $K > 20$ shows significant accuracy degradation.

TABLE III: MAE results across datasets (best configuration per model shown in bold).

		No Retrain			Fixed ($K = 10$)		
Dataset	Model	MAE	SMAPE	Retrains	MAE	SMAPE	Retrains
Electricity	LightGBM	1494.5	71.6	1	**119.5**	**6.0**	533
Electricity	TSMixer	872.3	42.3	1	633.5	26.5	533
Electricity	Random Forest	1811.4	86.6	1	153.6	9.6	533
Electricity	LSTM	2173.4	105.7	1	820.4	37.2	533
Brent	**TSMixer**	**1.81**	**3.8**	1	**1.51**	**3.3**	5
Brent	LightGBM	19.1	31.6	1	7.3	12.7	5
Brent	Random Forest	21.6	35.6	1	8.1	14.2	5
WTI	**TSMixer**	**1.87**	**4.1**	1	**1.37**	**3.2**	5
WTI	LightGBM	18.2	31.7	1	5.9	11.1	5
WTI	Random Forest	20.2	34.8	1	6.8	12.9	5
Traffic	LightGBM	212.7	9.8	1	**194.6**	**9.3**	198
Traffic	Random Forest	344.2	14.1	1	329.1	13.7	198

TABLE IV: Selected Pareto-optimal configurations.

Model + Strategy	MAE	Retrains	Notes
LightGBM + Fixed	119.5	533	Best on electricity
TSMixer + Fixed	1.51	5	Best on Brent/WTI
TSMixer + No Retrain	1.81	1	Zero-cost on Brent
Naïve + No Retrain	2219.3	0	Zero-cost baseline

TABLE V: Retraining strategy comparison on LightGBM model.

Dataset	Strategy	MAE	Retrains
Electricity	No Retrain	1494.5	0
Electricity	Fixed $K = 10$	**119.5**	533
Electricity	Adaptive	210.4	519
Traffic	No Retrain	212.7	0
Traffic	Fixed $K = 10$	**194.6**	198
Traffic	Adaptive	190.9	494
Brent	No Retrain	19.1	0
Brent	Fixed $K = 10$	7.3	5
Brent	Adaptive	**4.0**	11

TABLE VI: Adaptive vs Fixed retraining: Cases where Adaptive outperforms Fixed by >10%.

Dataset	Model	Adaptive MAE	Fixed MAE	Improvement	Retrain Savings
Brent	LightGBM	4.00	7.27	**45.0%**	+6
Brent	RF	4.61	8.13	**43.2%**	+6
WTI	LightGBM	3.61	5.85	**38.3%**	+6
WTI	RF	4.18	6.75	**38.1%**	+6
Electricity	TSMixer	557.79	633.49	**11.9%**	−149

TABLE VII: Ablation study: Fixed retraining interval K on Electricity/LightGBM.

K	Retrains	MAE	SMAPE
1	5330	95.2	5.1
5	1066	105.8	5.5
10	533	119.5	6.0
20	267	145.2	7.2
50	107	198.5	9.8
No Retrain	1	1494.5	71.6

VII. Discussion

A. Implications for Practitioners

Our central finding is that maintenance policy selection is often as consequential as model architecture choice. On electricity data, switching from no-retrain to fixed-interval with LightGBM reduces MAE by 92%, while replacing LightGBM with TSMixer under no-retrain reduces MAE by only 42%. Conversely, on Brent/WTI, TSMixer without retraining (MAE=1.81) dramatically outperforms LightGBM with fixed retraining (MAE=7.27).

Practitioners must therefore match model selection to data characteristics: gradient boosting excels on volatile, pattern-rich data; feature-engineered linear models excel on periodic financial series.

B. Limitations

Our study has several limitations:

- **Single replication**: Results are based on single experimental runs with seed 42. Future work should include multiple replications with different random seeds to compute statistical significance.
- **Adaptive strategy sensitivity**: The drift-triggered adaptive retraining strategy shows varying effectiveness across models. While it triggers successfully for TSMixer and achieves strong results on financial data (1.39 MAE on Brent vs 1.81 no-retrain), it requires threshold tuning for optimal performance. The number of retrains varies significantly: 519 for electricity/lgbm, 494 for traffic/lgbm, but only 4 for traffic/naive.
- **No synthetic drift injection**: Experiments rely on natural drift in real-world datasets. Controlled experiments with injected drift would provide more systematic evaluation of detection methods.
- **Dataset scope**: Evaluation on 4 datasets limits generalizability. Additional domains (healthcare, IoT) should be explored.
- **TSMixer implementation**: While achieving strong results, this implementation uses Ridge Regression with temporal

features rather than the original MLP-based TSMixer architecture, which requires GPU resources.

C. Future Work

Planned extensions include: multiple replications with statistical testing; automated threshold tuning via Bayesian optimisation; probabilistic forecasting models and metrics; integration of Transformer-based architectures; and a public leaderboard.

VIII. Conclusion

We have presented DriftBench-TS, a deployment-oriented benchmark and open-source framework for evaluating time series forecasting models under distribution shift. By embedding realistic deployment simulation, three configurable maintenance policies, and drift-correlated monitoring into a unified evaluation pipeline, DriftBench-TS enables systematic study of accuracy–robustness–cost trade-offs.

Our comprehensive experimental study across four public datasets and six model families reveals that model rankings are dataset-dependent: LightGBM with fixed retraining achieves 92% improvement on volatile energy data, while TSMixer outperforms all other models by 95% on financial/oil price data. Fixed-interval retraining provides consistent improvements; adaptive strategies require domain-specific threshold tuning.

These findings carry direct practical implications: practitioners must match model selection to data characteristics and deployment requirements. The full implementation and configuration files are released at https://github.com/madhan-karthikeyan/driftbench-ts under an open-source licence.

Acknowledgements

This work was conducted at Vellore Institute of Technology, Vellore.